CS509 Laboratory Submission

PG Software Lab M.Tech AI (2026-27)

Assignment Name:	Assignment 01 - BFS, DFS and SSSP
Students / Pair:	2026AIM1004 - Arshdeep Singh 2026AIM1007 - Ishtveer Singh Billing
Environment:	C++ Language | g++ compiler | GNU Make

1. Objective

- Program and execute Breadth-First Search (BFS), Depth-First Search (DFS), and Single-Source Shortest Path (SSSP) algorithms.
- Record and evaluate their execution times across various graph scales and input sizes.

2. Algorithm & Implementation Approach

- **Breadth-First Search (BFS):** Implemented using a standard Queue data structure.
- **Depth-First Search (DFS):** Implemented using a Stack data structure.
- **Single-Source Shortest Path (SSSP):** Implemented using Dijkstra's algorithm with a Min-Priority Queue (optimized for non-negative graph edge weights).
- **CSR Conversion:** Input adjacency list files are preprocessed and converted into Compressed Sparse Row (CSR) format (CsrGraph struct using csr.cpp / csr.h). Algorithms operate directly on the CSR representation.

3. Test Cases and Result Table

Algorithm	Test File	Vertices	Edges	Input Type	Src	Output File	Time	Status
BFS	unweighted10.txt	10	10	Unweighted CSR	0	output_bfs_10.txt	0 ms	**Pass**
BFS	unweighted100.txt	100	100	Unweighted CSR	0	output_bfs_100.txt	0 ms	**Pass**
BFS	unweighted10000.txt	10,000	10,000	Unweighted CSR	0	output_bfs_10000.txt	1 ms	**Pass**
BFS	unweighted50000.txt	50,000	50,000	Unweighted CSR	0	output_bfs_50000.txt	13 ms	**Pass**
BFS	unweighted100000.txt	100,000	100,000	Unweighted CSR	0	output_bfs_100000.txt	26 ms	**Pass**
DFS	unweighted10.txt	10	10	Unweighted CSR	0	output_dfs_10.txt	0 ms	**Pass**
DFS	unweighted100.txt	100	100	Unweighted CSR	0	output_dfs_100.txt	0 ms	**Pass**
DFS	unweighted10000.txt	10,000	10,000	Unweighted	0	output_dfs_10000.txt	3 ms	**Pass**

				CSR				
DFS	unweighted50000.txt	50,000	50,000	Unweighted CSR	0	output_dfs_50000.txt	17 ms	**Pass**
DFS	unweighted100000.txt	100,000	100,000	Unweighted CSR	0	output_dfs_100000.txt	34 ms	**Pass**
SSSP	sssp10.txt	10	10	Weighted CSR	0	output_sssp_10.txt	0 ms	**Pass**
SSSP	sssp100.txt	100	100	Weighted CSR	0	output_sssp_100.txt	0 ms	**Pass**
SSSP	sssp10000.txt	10,000	10,000	Weighted CSR	0	output_sssp_10000.txt	19 ms	**Pass**
SSSP	sssp50000.txt	50,000	50,000	Weighted CSR	0	output_sssp_50000.txt	113 ms	**Pass**
SSSP	sssp100000.txt	100,000	100,000	Weighted CSR	0	output_sssp_100000.txt	231 ms	**Pass**

4. Complexity Analysis

Breadth-First Search (BFS) & Depth-First Search (DFS)

- **Time Complexity:** **O(V + E)**, where V is the number of vertices and E is the number of edges. Every vertex and edge is processed exactly once.
- **Space Complexity:** **O(V + E)** for storing graph data in CSR format. Auxiliary space for Queue/Stack and visited arrays requires O(V) space. Overall space complexity is O(V + E).

Single-Source Shortest Path (SSSP - Dijkstra's Algorithm)

- **Time Complexity:** **O((V + E) log V)**. Using a min-priority queue allows extracting minimum distance vertices in O(log V) time and relaxing edges in O(log V) time.
- **Space Complexity:** **O(V + E)** to store the weighted graph in CSR format, plus O(V) auxiliary space for tracking distances and maintaining the priority queue.